

Práctica 1

Grupo Q

Marc Martínez Arias, Pedro Barros Bobadilla

[URL del repositorio](#)

En primer lugar, vamos a desarrollar los puntos necesarios para crear la simulación del mundo del Lejano Oeste. Con este mundo podremos comenzar a crear el proyecto basado en la búsqueda de las pepitas de oro. También importaremos las distintas librerías necesarias para el resto del proyecto.

```
In [1]: # Importamos las distintas librerías
import math
from threading import Thread, Lock, Semaphore
import time
import matplotlib.pyplot as plt
```

Ahora vamos a desarrollar todo el programa en el mismo código. Dado que lo vamos a definir como una clase vamos a implementarlo todo en la misma celda donde se recopilará el init donde definamos todos los tipos y las funciones de los distintos tipos de objetos que creemos.

```
In [ ]: class CentralOperaciones:

    # Definimos el init donde se recogen todos los tipos de objetos. En s
    def __init__(self):

        # Los hashes dorados, que actuan como la recompensa de oro:
        self.coordenadasDoradas = []
        self.cerrojoCoords = Lock()

        # Exploración por anillos:
        self.radio_actual = 1
        self.cuadrante_actual = 0
        self.lockExploradas = Lock()

        # Para los graficos:
        self.hist_tiempo = []
        self.hist_pepitas = []
        self.hist_viajeras = []

        # Los trabajadores:
        self.exploradoras = []
        self.recolectoras = []
        self.recolectorasViajeras = []
        self.calculadorasRuta = []

        # El sistema económico:
        self.numPepitas = 0
        self.lockPepitas = Lock()
```

```

        # Las distintas rutas capturadas:
        self.rutas = []
        self.lockRutas = Lock()

        # Las carretillas que recogen el oro, definido mediante un semáforo
        self.semCarretillas = Semaphore(0)
        self.numCarretillas = 0

# Funciones de la economía de la clase:

    # Definimos la función que aumenta la economía:
    def __añadirPepita(self):

        with self.lockPepitas:
            self.numPepitas += 1

    # Función que añade el hash dorado:
    def __addCoordenadaDorada(self, coord):

        with self.cerrojoCoords:
            self.coordenadasDoradas.append(coord)

    # Función que comprueba el hash dorado, como en la P0
    def __comprobar_hash(self, coord):

        # probabilidad de aparición de oro
        if hash(coord) % 10000000 == 0:
            self.__addCoordenadaDorada(coord)

# Funcionamiento de las exploradoras:

    # Definimos la forma en la que la exploradora explora, utilizando man
    def __siguienteZona(self):

        with self.lockExploradas:

            radio = self.radio_actual
            cuadrante = self.cuadrante_actual

            self.cuadrante_actual += 1

            if self.cuadrante_actual == 4:
                self.cuadrante_actual = 0
                self.radio_actual += 1

            return radio, cuadrante

    # Función que explora en forma de anillo, implementada en la anterior
    def __explorar_anillo(self, r, cuadrante):

        NW, NE, SE, SW = 0, 1, 2, 3

        # Casos según el cuadrante
        if cuadrante == NW:
            # izquierda
            for x in range(0, -r-1, -1):
                self.__comprobar_hash((x, 0))
            # arriba

```

```

        for y in range(1, r+1):
            self.__comprobar_hash((-r, y))
        # derecha
        for x in range(-r+1, 1):
            self.__comprobar_hash((x, r))
        # abajo
        for y in range(r-1, 0, -1):
            self.__comprobar_hash((0, y))

    elif cuadrante == NE:
        # arriba
        for y in range(0, r+1):
            self.__comprobar_hash((0, y))
        # derecha
        for x in range(1, r+1):
            self.__comprobar_hash((x, r))
        # abajo
        for y in range(r-1, -1, -1):
            self.__comprobar_hash((r, y))
        # izquierda
        for x in range(r-1, 0, -1):
            self.__comprobar_hash((x, 0))

    elif cuadrante == SE:
        # derecha
        for x in range(0, r+1):
            self.__comprobar_hash((x, 0))
        # abajo
        for y in range(-1, -r-1, -1):
            self.__comprobar_hash((r, y))
        # izquierda
        for x in range(r-1, -1, -1):
            self.__comprobar_hash((x, -r))
        # arriba
        for y in range(-r+1, 0):
            self.__comprobar_hash((0, y))

    elif cuadrante == SW:
        # abajo
        for y in range(0, -r-1, -1):
            self.__comprobar_hash((0, y))
        # izquierda
        for x in range(-1, -r-1, -1):
            self.__comprobar_hash((x, -r))
        # arriba
        for y in range(-r+1, 1):
            self.__comprobar_hash((-r, y))
        # derecha
        for x in range(-r+1, 0):
            self.__comprobar_hash((x, 0))

# Función con la que obtenemos el cuadrante:
def obtener_cuadrante(self, coord):
    x, y = coord

    if x >= 0 and y >= 0:
        return "NE"
    elif x < 0 and y >= 0:
        return "NW"
    elif x >= 0 and y < 0:

```

```

        return "SE"
    else:
        return "SW"

# La función main de la exploradora, con la que realmente explora las
def __exploradora(self):
    while True:
        radio, cuadrante = self.__siguienteZona()
        self.__explorar_anillo(radio, cuadrante)
        time.sleep(0.002)

# Función con la que se añade la exploradora a la explotación minera,
def añadirExploradora(self):
    t = Thread(target=self.__exploradora)
    t.start()
    self.exploradoras.append(t)

# Apartado de los hashes dorados o pepitas doradas:

# Función que gestiona las coordenadas o hashes dorados
def gestionarCoordenadasDoradas(self, coord):
    time.sleep(8.64)
    with self.cerrojoCoords:
        if coord not in self.coordenadasDoradas:
            self.coordenadasDoradas.append(coord)

# Definimos los recolectores de las pepitas:
def __recolectora(self):
    while True:
        coord = None
        with self.cerrojoCoords:
            if self.coordenadasDoradas:
                coord = min(
                    self.coordenadasDoradas,
                    key=lambda c: abs(c[0]) + abs(c[1])
                )
                self.coordenadasDoradas.remove(coord)
        if coord is None:
            time.sleep(0.001)
            continue
        distancia = abs(coord[0]) + abs(coord[1])
        time.sleep(distancia * 0.01)
        t = Thread(target=self.gestionarCoordenadasDoradas, args=(coord,))
        t.daemon = True
        t.start()
        time.sleep(distancia * 0.01)
        self.__añadirPepita()

# Añadimos el recolector, igual que antes. Añadimos otro hilo, que en
def añadirRecolectora(self):
    t = Thread(target=self.__recolectora)
    t.start()
    self.recolectoras.append(t)

# Función que calcula las rutas exploradas por las exploradoras
def __calculadoraRuta(self):
    while True:
        cuadrantes = ["NW", "NE", "SE", "SW"]
        with self.lockExploradas:
            cuadrante = cuadrantes[self.cuadrante_actual]

```

```

        self.cuadrante_actual = (self.cuadrante_actual + 1) % 4
    with self.cerrojoCoords:
        coords = [
            c for c in self.coordenadasDoradas
            if self.obtener_cuadrante(c) == cuadrante
        ]

    # las ordenamos por cercanía a la base (0,0) y nos quedamos con 1
    coords.sort(key=lambda c: abs(c[0]) + abs(c[1]))
    coords = coords[:30]
    if len(coords) < 2:
        time.sleep(0.05)
        continue
    ruta = []
    posicion = (0, 0)
    while coords and len(ruta) < 6:
        nearest = min(
            coords,
            key=lambda c: abs(c[0] - posicion[0]) + abs(c[1] - po
        )
        ruta.append(nearest)
        coords.remove(nearest)
        posicion = nearest
    if len(ruta) < 3:
        continue
    with self.lockRutas:
        if len(self.rutas) < 200:
            self.rutas.append(ruta)
    time.sleep(0.05)

# Función que calcula las rutas a explorar:
def añadirCalculadoraRuta(self):
    t = Thread(target=self.__calculadoraRuta)
    t.start()
    self.calculadorasRuta.append(t)

# Función que gestiona los recolectores y viajeros:
def __recolectoraViajera(self):
    while True:
        # esperar carretilla
        self.semCarretillas.acquire()
        ruta = None
        with self.lockRutas:
            if self.rutas:
                ruta = self.rutas.pop(0)
        if ruta is None:
            self.semCarretillas.release()
            time.sleep(0.05)
            continue
        posicion = (0, 0)
        tiempo_total = 0
        for coord in ruta:
            distancia = abs(coord[0] - posicion[0]) + abs(coord[1] -
            time.sleep(distancia * 0.01)
            tiempo_total += distancia
            posicion = coord

        # volver a base
        distancia = abs(posicion[0]) + abs(posicion[1])
        time.sleep(distancia * 0.01)

```

```

tiempo_total += distancia

# añadir pepitas (1 por cada punto de la ruta)
with self.lockPepitas:
    self.numPepitas += len(ruta)

    # salario viajera (1 cada 10 días)
    salario = math.ceil(tiempo_total / 864000)
    self.numPepitas -= salario

# liberar carretilla
self.semCarretillas.release()
print("Viajera completó ruta:", len(ruta), "pepitas")

# Añadimos la recolectora a la viajera para que recoja lo explorado:
def añadirRecolectoraViajera(self):
    t = Thread(target=self.__recolectoraViajera)
    t.start()
    self.recolectorasViajeras.append(t)

# Aquí definimos el gestor económico que tratará las pepitas:
def __gestorEconomico(self):
    dias = 0
    while True:
        time.sleep(1)
        dias += 1
        # estado actual
        with self.lockPepitas:
            pepitas = self.numPepitas
        with self.cerrojoCoords:
            coords = len(self.coordenadasDoradas)

        # Escalamos las recolectoras: máximo 4 recolectoras hasta tener
        if len(self.recolectorasViajeras) == 0:

            if coords > len(self.recolectoras) * 5000:
                print("Añadiendo recolectora")
                self.añadirRecolectora()
            trabajadores = len(self.exploradoras) + len(self.recolectoras)

        # Pagamos los salarios a los trabajadores:
        if dias % 30 == 0:
            coste = trabajadores
            with self.lockPepitas:
                if self.numPepitas >= coste:
                    self.numPepitas -= coste
                    print("Salarios pagados:", coste)
                else:
                    print(" No hay pepitas suficientes para salarios")

        # Creamos una reserva guardando pepitas para poder pagar el salario
        reserva_salarios = trabajadores * 2

        # Ahora creamos el flujo:
        # PRIMERA VIAJERA (OBJETIVO PRINCIPAL)
        if self.numCarretillas == 0:
            with self.lockPepitas:

                # FORZAR PRIMERA CARRETILLA
                if self.numPepitas >= 2:

```

```

        self.numPepitas -= 2
        self.numCarretillas += 1
        self.semCarretillas.release()
        print(self.coordenadasDoradas)

#ACTIVAR RECOLECTORA VIAJERA
if self.numCarretillas > 0:
    if len(self.recolectorasViajeras) < self.numCarretillas:
        print("Añadiendo recolectora viajera")
        self.añadirRecolectoraViajera()

#ESCALADO PROGRESIVO
limite_viajeras = 1
if pepitas > 5:
    limite_viajeras = 2
if pepitas > 10:
    limite_viajeras = 3
if pepitas > 20:
    limite_viajeras = 4
if pepitas > 50:
    limite_viajeras = 5
if pepitas > 100:
    limite_viajeras = 7
if pepitas > 300:
    limite_viajeras = 10
if pepitas > 800:
    limite_viajeras = 20

# COMPRA DE CARRETILLAS
with self.lockPepitas:
    if (
        self.numPepitas >= 3 + reserva_salarios
        and self.numCarretillas < limite_viajeras
    ):
        self.numPepitas -= 3
        self.numCarretillas += 1
        self.semCarretillas.release()
        print("Carretilla comprada:", self.numCarretillas)

# Iniciamos el gestor económico que controla la economía de la explot
def iniciarGestorEconomico(self):
    t = Thread(target=self.__gestorEconomico)
    t.start()

```

Ahora que hemos creado todo el código de la práctica vamos a desarrollar un ejemplo para comprobar su funcionamiento.

```

In [ ]: if __name__ == "__main__":
        central = CentralOperaciones()
        central.añadirExploradora()
        central.añadirRecolectora()
        central.añadirCalculadoraRuta()
        central.iniciarGestorEconomico()

# SOLO UNA EJECUCIÓN
inicio = time.time()
duracion = 200
while time.time() - inicio < duracion:
    tiempo_actual = time.time()

```

```

with central.lockPepitas:
    pepitas = central.numPepitas
    viajeras = len(central.recolectorasViajeras)
    central.hist_tiempo.append(tiempo_actual)
    central.hist_pepitas.append(pepitas)
    central.hist_viajeras.append(viajeras)
    time.sleep(5)
with central.lockPepitas:
    print("Pepitas:", central.numPepitas)
    print("Exploradoras:", len(central.exploradoras))
    print("Recolectoras:", len(central.recolectoras))
    print("Recolectoras viajeras:", len(central.recolectorasViajeras))
    print("Carretillas:", central.numCarretillas)
    print("Rutas pendientes:", len(central.rutas))

with central.cerrojoCoords:
    print("Coords doradas:", len(central.coordenadasDoradas))
    print("-----")

# Gráfico
print("Simulación terminada, generando gráficos...")
tiempo = [t - central.hist_tiempo[0] for t in central.hist_tiempo]
plt.figure()
plt.plot(tiempo, central.hist_pepitas)
plt.title("Evolución de Pepitas")
plt.xlabel("Tiempo")
plt.ylabel("Pepitas")
plt.figure()
plt.plot(tiempo, central.hist_viajeras)
plt.title("Recolectoras Viajeras")
plt.xlabel("Tiempo")
plt.ylabel("Número de viajeras")
plt.show()

```


Pepitas: 1
Exploradoras: 1
Recolectoras: 1
Recolectoras viajeras: 0
Carretillas: 0
Rutas pendientes: 0
Coords doradas: 0

Pepitas: 1
Exploradoras: 1
Recolectoras: 1
Recolectoras viajeras: 0
Carretillas: 0
Rutas pendientes: 0
Coords doradas: 3

Pepitas: 1
Exploradoras: 1
Recolectoras: 1
Recolectoras viajeras: 0
Carretillas: 0
Rutas pendientes: 0
Coords doradas: 5

Pepitas: 1
Exploradoras: 1
Recolectoras: 1
Recolectoras viajeras: 0
Carretillas: 0
Rutas pendientes: 21
Coords doradas: 10

Pepitas: 1
Exploradoras: 1
Recolectoras: 1
Recolectoras viajeras: 0
Carretillas: 0
Rutas pendientes: 96
Coords doradas: 12

[(-629, -354), (570, 817), (-883, -914), (677, -1070), (209, 1351), (824, 1377), (824, 1377), (977, -1410), (-1137, -1474), (1494, 1681), (402, -584), (-1807, -1778)]

Añadiendo recolectora viajera

Pepitas: 1
Exploradoras: 1
Recolectoras: 1
Recolectoras viajeras: 1
Carretillas: 1
Rutas pendientes: 176
Coords doradas: 14

No hay pepitas suficientes para salarios

Pepitas: 1
Exploradoras: 1
Recolectoras: 1
Recolectoras viajeras: 1
Carretillas: 1
Rutas pendientes: 200
Coords doradas: 17

Pepitas: 1
Exploradoras: 1
Recolectoras: 1
Recolectoras viajeras: 1
Carretillas: 1
Rutas pendientes: 200
Coords doradas: 25

Pepitas: 1
Exploradoras: 1
Recolectoras: 1
Recolectoras viajeras: 1
Carretillas: 1
Rutas pendientes: 200
Coords doradas: 29

Pepitas: 3
Exploradoras: 1
Recolectoras: 1
Recolectoras viajeras: 1
Carretillas: 1
Rutas pendientes: 200
Coords doradas: 31

Pepitas: 3
Exploradoras: 1
Recolectoras: 1
Recolectoras viajeras: 1
Carretillas: 1
Rutas pendientes: 200
Coords doradas: 36

Pepitas: 3
Exploradoras: 1
Recolectoras: 1
Recolectoras viajeras: 1
Carretillas: 1
Rutas pendientes: 200
Coords doradas: 48

Salarios pagados: 2
Pepitas: 1
Exploradoras: 1
Recolectoras: 1
Recolectoras viajeras: 1
Carretillas: 1
Rutas pendientes: 200
Coords doradas: 51

```
-----  
-  
KeyboardInterrupt                                Traceback (most recent call las  
t)  
Cell In[3], line 27  
    24 central.hist_pepitas.append(pepitas)  
    25 central.hist_viajeras.append(viajeras)  
--> 27 time.sleep(5)  
    29 with central.lockPepitas:  
    30     print("Pepitas:", central.numPepitas)  
  
KeyboardInterrupt:
```

Viajera completó ruta: 4 pepitas
Salarios pagados: 2
Carretilla comprada: 2
Añadiendo recolectora viajera
Viajera completó ruta: 4 pepitas
Salarios pagados: 2
Salarios pagados: 2
Viajera completó ruta: 4 pepitas
Carretilla comprada: 3
Viajera completó ruta: 4 pepitas
Añadiendo recolectora viajera
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Salarios pagados: 2
Viajera completó ruta: 4 pepitas
Carretilla comprada: 4
Viajera completó ruta: 4 pepitas
Añadiendo recolectora viajera
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 4 pepitas
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Carretilla comprada: 5
Viajera completó ruta: 3 pepitas
Añadiendo recolectora viajera
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 3 pepitas

Viajera completó ruta: 3 pepitas
Viajera completó ruta: 4 pepitas
Salarios pagados: 2
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Carretilla comprada: 6
Añadiendo recolectora viajera
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Carretilla comprada: 7
Añadiendo recolectora viajera
Salarios pagados: 2
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 4 pepitas
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 5 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 5 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 5 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 5 pepitas

Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 5 pepitas
Viajera completó ruta: 5 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 5 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 4 pepitas
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 5 pepitas
Viajera completó ruta: 5 pepitas
Salarios pagados: 2
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 5 pepitas
Viajera completó ruta: 4 pepitas
Salarios pagados: 2
Viajera completó ruta: 5 pepitas
Viajera completó ruta: 4 pepitas
Salarios pagados: 2
Viajera completó ruta: 5 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 5 pepitas
Viajera completó ruta: 4 pepitas
Salarios pagados: 2
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 4 pepitas
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Carretilla comprada: 8
Añadiendo recolectora viajera
Carretilla comprada: 9
Añadiendo recolectora viajera
Viajera completó ruta: 4 pepitas
Carretilla comprada: 10
Viajera completó ruta: 5 pepitas
Añadiendo recolectora viajera
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 5 pepitas

Viajera completó ruta: 4 pepitas
Viajera completó ruta: 4 pepitas
Salarios pagados: 2
Viajera completó ruta: 5 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 4 pepitas
Salarios pagados: 2
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 6 pepitas
Salarios pagados: 2
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 3 pepitas
Salarios pagados: 2
Viajera completó ruta: 3 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 6 pepitas
Viajera completó ruta: 6 pepitas
Salarios pagados: 2
Viajera completó ruta: 6 pepitas
Viajera completó ruta: 6 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 6 pepitas
Viajera completó ruta: 4 pepitas
Salarios pagados: 2
Viajera completó ruta: 6 pepitas
Viajera completó ruta: 6 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 6 pepitas
Viajera completó ruta: 4 pepitas
Salarios pagados: 2
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 6 pepitas
Salarios pagados: 2
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 6 pepitas
Salarios pagados: 2
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 6 pepitas
Viajera completó ruta: 4 pepitas
Viajera completó ruta: 6 pepitas
Viajera completó ruta: 4 pepitas
Salarios pagados: 2